Centro Federal de Educação Tecnológica de Minas Gerais
Professora: Poliana Corrêa

RELATÓRIO PRÁTICA 5 - LAOC
Banco de Registradores

ALUNOS:
GIOVANNI ESTEVAM DOS SANTOS
JOÃO VITOR LOPES FAGUNDES

Belo Horizonte - MG
2025

1. Introdução

A prática 5 (Banco de registradores), teve como foco principal construir a implementação de um banco de registradores. Este banco deve ser parametrizável e composto por 32 registradores, onde cada um possui capacidade de armazenamento de 32 bits. Para o seu funcionamento, é necessário que ele realize operações de leitura e escrita. A leitura precisa de duas portas de saída (RD1 e RD2) que mostram os dados dos registradores selecionados pelos endereços A1 e A2, e ela ocorre de forma assíncrona. Já a escrita utiliza uma porta de dados (WD3) para gravar o dado no registrador indicado pelo endereço A3. Além disso, a escrita é síncrona, dependendo da borda de subida do CLK e da habilitação do sinal de escrita (WE3).

2. Desenvolvimento

Nosso projeto da Prática 5 consiste na implementação de um Banco de Registradores, um componente digital essencial para unidades de processamento. O módulo implementa um conjunto de 32 registradores, onde cada um possui 32 bits de capacidade de armazenamento. A arquitetura do projeto foi focada em atender aos requisitos específicos de leitura e escrita do processador MIPS: a leitura de duas portas é realizada de forma assíncrona, enquanto a escrita em uma porta é síncrona, controlada por um sinal de clock e uma habilitação de escrita.

Descrição e Decisões de Projeto dos Módulos

1. Banco de Registradores: bancoRegistradores

Descrição: O módulo **bancoRegistradores** implementa um array de 32 registradores, onde cada um armazena 32 bits. Ele possui três portas de acesso principais: duas portas de saída para leitura de dados (Data1, Data2), que refletem o conteúdo dos registradores selecionados pelas entradas de endereço (Read1, Read2). Uma terceira porta é usada para a escrita, onde o dado de 32 bits em WriteData é gravado no registrador selecionado pelo endereço WriteReg. A escrita é uma operação síncrona, controlada pela borda de subida do clock e pelo sinal de habilitação RegWrite. O módulo também inclui uma lógica de inicialização que zera todos os 32 registradores no início da simulação.

Decisões de Projeto: Para este componente, tomamos várias decisões de implementação cruciais baseadas no enunciado e nas dicas fornecidas:

Armazenamento: Em vez de instanciar Flip-Flops individuais, optamos por utilizar o tipo de dados reg do Verilog para criar o array de registradores (reg [31:0] registradores [0:31];). Esta abordagem, sugerida pelo enunciado, é uma forma muito mais abstrata, legível e eficiente de descrever um banco de memória ou registradores para a ferramenta de síntese.

Inicialização: Para garantir um estado inicial conhecido e determinístico, decidimos usar um bloco initial. Dentro deste bloco, um laço for é utilizado para varrer todos os 32 endereços (de 0 a 31) e atribuir o valor zero a cada registrador. Esta decisão segue a dica fornecida no enunciado.

Leitura Assíncrona: Para cumprir o requisito de leitura assíncrona , as saídas de leitura (Data1 e Data2) foram implementadas com atribuições contínuas (assign). Elas são posicionadas fora de qualquer bloco sequencial (always), o que gera uma lógica puramente combinacional. Esta decisão garante que, assim que os endereços de leitura (Read1 ou Read2) mudam, as saídas são atualizadas imediatamente com o novo dado armazenado, sem a necessidade de esperar por um pulso de clock.

Escrita Síncrona: Para a escrita, a decisão foi usar um bloco always sensível à borda de subida do clock (@(posedge clock)). Esta é a implementação direta do requisito de que a escrita seja síncrona. Dentro deste bloco, a operação de escrita só é executada se o sinal de habilitação (RegWrite) estiver em nível lógico '1' (if(RegWrite)), conforme especificado. Optamos por usar uma atribuição non-blocking (<=), que é a prática padrão recomendada para descrever hardware sequencial (baseado em registradores), pois evita condições de corrida e reflete corretamente o comportamento de um registrador que atualiza seu valor apenas no pulso do clock.

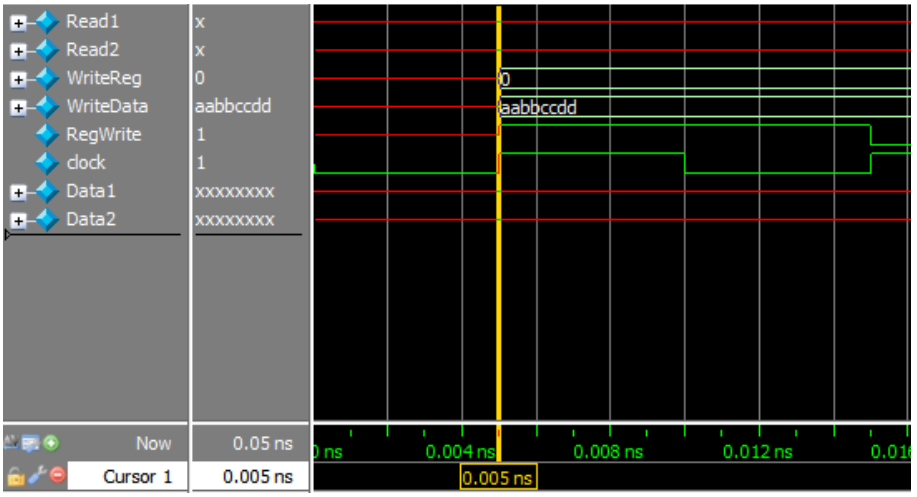
2. Módulo de Teste: bancoRegistradores_tb

Descrição: O **bancoRegistradores_tb** é um módulo de testbench, cujo único propósito é simular e verificar o funcionamento do módulo **bancoRegistradores**. Ele não é sintetizável (não se transforma em hardware real), servindo apenas para criar os estímulos de entrada (como clock, dados e endereços) e observar se as saídas do módulo sob teste correspondem ao comportamento esperado.

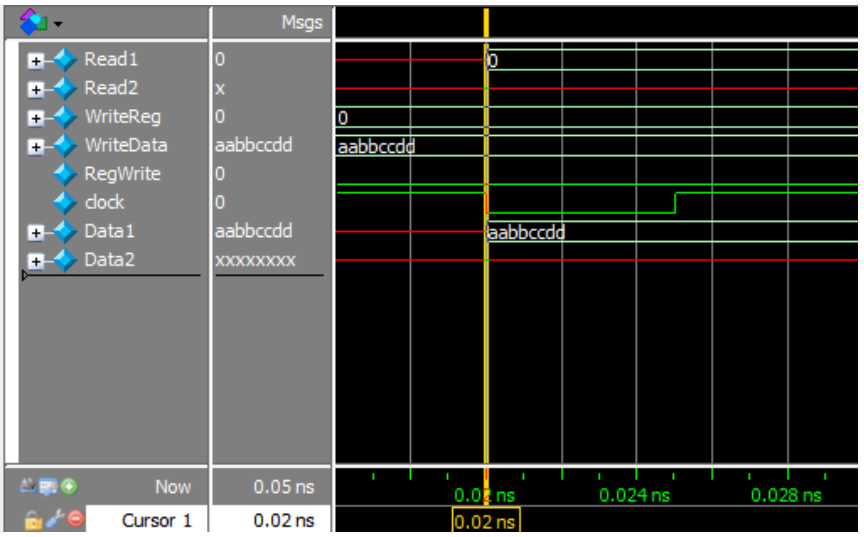
Funcionamento:

Instanciação e Clock: Primeiramente, ele instancia o módulo **bancoRegistradores**. Em paralelo, utiliza um bloco initial para definir o valor inicial do clock como 0 e um bloco always para gerar um sinal de clock periódico, que inverte seu valor a cada 5 unidades de tempo (#5 clock = ~clock;).

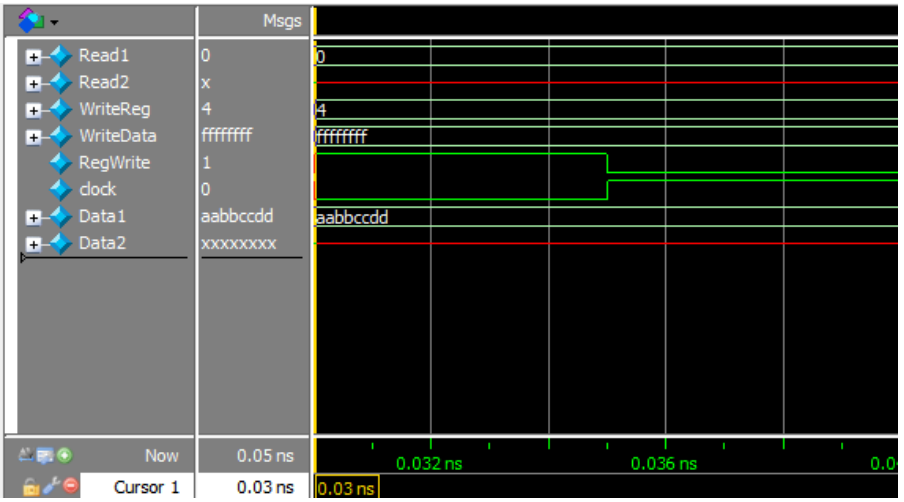
3. Simulações



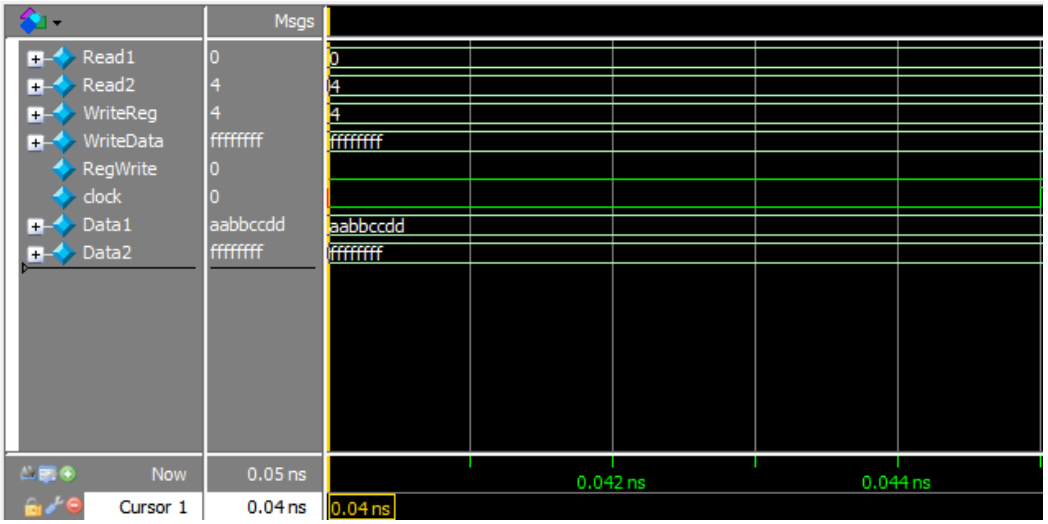
No início da simulação, RegWrite é setado em 1 e, quando está na borda de subida do clock, escreve aabbccdd no endereço 0 do registrador.



Após isso, Read1 recebe o endereço 0 e é feita a leitura do conteúdo do registrador nessa posição, salvando em Data1 (aabbccdd).



Na segunda escrita da simulação, RegWrite é novamente setado em 1 e, quando está na borda de subida do clock, escreve ffffffff no endereço 4 do registrador.



Por último, Read2 recebe o endereço 4 e é feita a leitura do conteúdo do registrador nessa posição, salvando em Data2 (ffffffff).